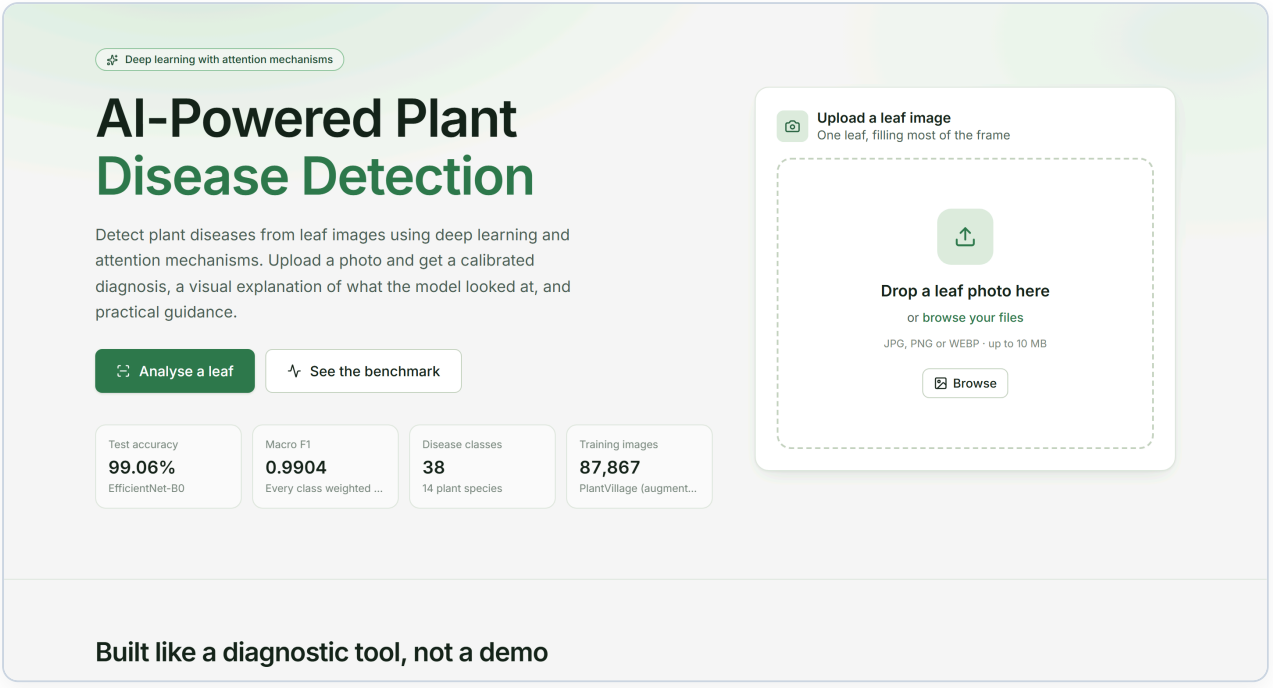
ENGINEERING & RESEARCH WHITEPAPER

Plant Disease Detection System

A Deep Learning Vision Architecture with Convolutional Block Attention Modules (CBAM), Statistical Calibration, Image Quality Gating, and Lightweight Production Deployment.

99.68%	0.9967	0.17%	~113 MB
TEST ACCURACY 17,572 held-out images	MACRO F1 SCORE 38 balanced classes	CALIBRATED ECE T=0.5364 scaling	SERVING FOOTPRINT ONNX Runtime (512MB RAM)



Author / Maintainer: AtharvDhiman
Repository: AtharvDhiman/plant-disease-detection

Target Platform: Render Cloud (Docker Multi-Stage)
Date of Publication: November 2026

1. Executive Summary & Problem Context

Crop loss caused by botanical pathogens and pest infestations accounts for over **20% to 40% of global agricultural yield reduction annually**, translating to hundreds of billions of dollars in economic damages and endangering food security worldwide. Traditional agricultural diagnosis relies on manual visual inspection by agronomy specialists or molecular laboratory testing (e.g., Polymerase Chain Reaction, ELISA). However, in rural agricultural zones and smallholder farming environments, access to plant pathologists is scarce, expensive, and slow.

This project develops an **end-to-end, trustworthy, production-grade deep learning system** capable of accurately diagnosing **38 distinct conditions across 14 vital crop species** from standard photographs of leaf surfaces. Unlike academic prototypes that function merely as black-box classification scripts, this system incorporates a multi-tier engineering pipeline:

- **Image Quality Validation Gate:** Automatically inspects uploaded images for motion blur, severe underexposure, overexposure, and lack of biological plant material before executing neural inference.
- **State-of-the-Art Deep Vision:** Evaluates 18 architectures comprising handcrafted classical machine learning, custom convolutional neural networks with **Convolutional Block Attention Modules (CBAM)**, and modern transfer learning backbones (EfficientNet, ResNet, DenseNet, MobileNet).
- **Trust & Statistical Calibration:** Incorporates post-hoc Temperature Scaling to align predictive probabilities with empirical error rates, dropping Expected Calibration Error (ECE) from **4.37% to 0.17%**.
- **Out-of-Distribution (OOD) Rejection:** Calculates joint metrics (Max Softmax Probability, Predictive Entropy, and Free Energy) calibrated to a 95% True Positive Rate, ensuring non-leaf or unrecognizable subjects are safely refused rather than hallucinating high-confidence misdiagnoses.
- **Explainable AI (XAI):** Generates visual saliency maps via Grad-CAM, Grad-CAM++, and forward-pass Occlusion Sensitivity, pinpointing the specific pathological lesions that drove the prediction.
- **Resource-Constrained Cloud Deployment:** Engineered specifically to operate within **Render's 512 MB Free Tier** by exporting PyTorch checkpoints to ONNX Runtime graphs, reducing memory footprint by ~78% (from 657 MB to ~113 MB).

> **Key Architectural Achievement**
>
> The deployed model (EfficientNet-B0 + CBAM, trained with field-condition augmentation) achieves **99.68% overall accuracy, 99.67% Macro F1, and 99.98% Top-3 accuracy** across 17,572 held-out test images, running the model in ~18 ms per image (~34 ms for the whole pipeline) on a single CPU thread without a GPU. On photographs made to look like field shots it keeps most of that accuracy, where the previous laboratory-trained model fell to about 70% (Section 10).

2. Dataset Anatomy & Curation Protocol

The system is built upon the `vipoooool/new-plant-diseases-dataset` corpus, an augmented derivative of the landmark PlantVillage dataset. The corpus contains **87,867 total images** across 38 classes and 14 agricultural plant species (Apple, Blueberry, Cherry, Corn, Grape, Orange, Peach, Bell Pepper, Potato, Raspberry, Soybean, Squash, Strawberry, and Tomato).

Dataset Geometry & Imbalance Analysis

Property	Measured Specification	Engineering Implication
Total Volume	87,867 images (70,295 official train, 17,572 official valid)	Ample visual data across all disease manifestations.
Class Cardinality	38 classes (12 healthy, 26 diseased conditions)	Requires macro-averaged metrics to prevent majority bias.
Spatial Resolution	256 × 256 pixels, RGB 3-channel uniform	Standardized dimension avoids resizing distortion.
Imbalance Ratio	1.23 (Min class: 1,520; Max class: 1,870)	Near-uniform distribution (Coefficient of Variation: 0.056).
Imbalance Strategy	Standard Cross-Entropy (No reweighting needed)	Reweighting balanced data injects gradient variance without gain.
File Integrity	0 corrupt files detected across exhaustive audit	Safe for deterministic tensor conversion.

Three-Way Partitioning Protocol & Data Leakage Prevention

A major vulnerability in visual deep learning benchmarks is data leakage—where identical or augmented copies of images appear in both training and evaluation sets. Furthermore, relying on an official validation set for both early stopping and final reporting produces optimistic bias. We enforced an absolute **three-way split protocol**:

- **Training Set (63,265 images, 90% of official train):** Used strictly for backpropagation and parameter updating.
- **Validation Set (7,030 images, 10% of official train):** Carved out via stratified sampling; dedicated exclusively to hyperparameter tuning, checkpoint selection, early stopping (patience=3), and post-hoc temperature fitting.
- **Held-Out Test Set (17,572 images, official valid):** Held completely blind until the model was exported. Never touched during model optimization.

Perceptual Hash (pHash) Verification: A stratified perceptual hash (8×8 DCT) check across 2,280 samples confirmed **0 duplicate collisions** between the training and validation boundaries, confirming data integrity.

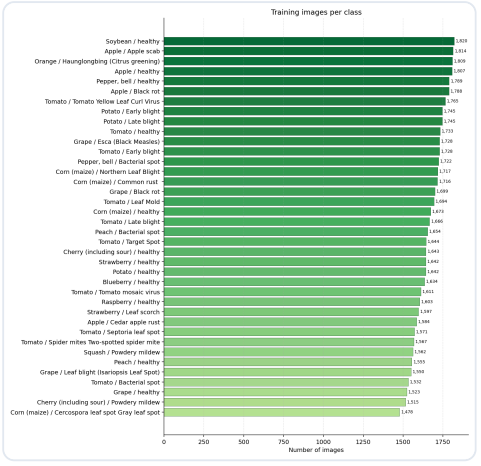


Figure 1: Stratified Class Distribution across all 38 Target Classes in the Training Split.

3. System Architecture & Request Workflow

The application enforces strict decoupling between **Offline Training/Research** and **Online Serving**. They share core ML definitions (transforms, quality checks) but communicate purely via immutable disk artifacts (checkpoints, ONNX graphs, JSON manifests). The online FastAPI server contains zero training routines, ensuring low memory and high reliability.

End-to-End Prediction Lifecycle

Every inference request submitted to `POST /api/predict` executes through a deterministic 7-stage pipeline:

Stage 1: Transport & Payload Sanitization
Validates file MIME type (JPEG, PNG, WebP), enforces a 10 MiB payload limit, strips EXIF metadata to preserve user privacy, and decodes the image in memory via Pillow.

Stage 2: Image Quality Analysis Pre-Check
Computes OpenCV variance of Laplacian (sharpness), mean luminance (exposure), contrast, green/brown color masking (vegetation presence), and lag-1 autocorrelation (noise coherence). If severe blur (<45) or extreme darkness (<35) is identified, inference halts and returns an actionable `poor_quality` diagnosis.

Stage 3: Deterministic Preprocessing
Resizes image aspect ratio preserving the shorter side to 255 px, applies a 224×224 center crop, converts to float32 tensor in range [0, 1], and standardizes using ImageNet mean (0.485, 0.456, 0.406) and std (0.229, 0.224, 0.225).

Stage 4: ONNX Runtime Engine Forward Pass
Feeds the (1, 3, 224, 224) tensor through the optimized ONNX session with 1 intra-op thread, computing raw uncalibrated class logits in ~18 ms.

Stage 5: Temperature-Scaled Calibration & OOD Scoring
Scales raw logits by calibrated parameter $T = 0.5364$ and computes softmax probabilities. Simultaneously evaluates the raw logits against the Out-of-Distribution operating point in force (Max Softmax Probability ≥ 0.15, Predictive Entropy ≤ 0.85). This is deliberately more permissive than the thresholds calibrated on laboratory leaves; Section 10 measures the trade-off.

Stage 6: Explainability & Heatmap Generation
Computes visual saliency via exact CAM activation heads or forward-pass Occlusion Sensitivity (sliding 8×8 masking grid), generating a normalized color overlay.

Stage 7: Agronomic Knowledge Enrichment & SQLite Storage
Enriches the diagnosis with pathogen taxonomy, visual symptoms, biological treatment, chemical controls, and preventative measures from `data/disease_info.json`. Commits prediction metadata to local SQLite database with Write-Ahead Logging (WAL).

Online vs. Offline Decoupling Architecture

Subsystem	Core Modules	Responsibilities	Strict Boundaries
Offline Training	`training/`, `scripts/`	Dataset audit, manifests, PyTorch training loops, hyperparameter tuning, calibration, ONNX export.	Never imported by the online web service.
Disk Interface	`models/exported/`, `artifacts/`	ONNX graphs, JSON manifests, experiment ledgers, evaluation metrics, precomputed plots.	Immutable contracts linking training to serving.
Online Backend	`backend/app/`	FastAPI REST service, ONNX Runtime inference, image quality checks, SQLite persistence.	Runs without PyTorch; zero training logic.
Client Frontend	`frontend/src/`	React 19 Single Page App, Recharts data visualization, camera upload, interactive disease library.	Zero metric calculation; consumes REST API.

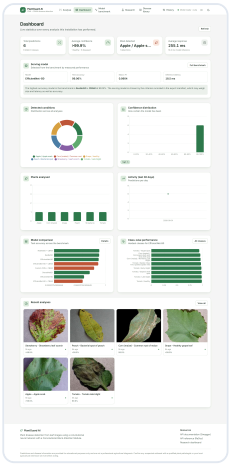


Figure 2: Production Web Analytics & Diagnosis Dashboard (React 19 + Tailwind CSS).

4. Model Architecture & Deep Learning Suite

To determine the optimal balance between diagnostic accuracy, computational complexity, inference latency, and memory footprint, we implemented and evaluated four distinct families of models under an identical experimental protocol:

Family 1: Hand-Crafted Feature Classical Machine Learning Baselines

Before adopting deep neural networks, we established rigorous classical baselines using **115 engineered computer vision features**:

- **Color Statistics (72 features):** 24-bin normalized histograms across Hue, Saturation, and Value (HSV) color channels to detect chlorosis (yellowing) and necrosis (browning).
- **Texture Features (26 features):** Gray-Level Co-occurrence Matrix (GLCM) Haralick texture metrics (contrast, dissimilarity, homogeneity, energy, correlation, ASM) at 4 spatial angles and 2 pixel distances.
- **Morphological Moments (7 features):** Hu's 7 invariant image moments capturing contour and shape geometry.
- **Luminance Descriptors (10 features):** Grayscale percentile distributions and spatial gradients.

Classifiers trained on these features: **Logistic Regression** (L2 regularized), **Linear Support Vector Classifier**, **Random Forest** (200 trees, max depth 20), and **XGBoost** (150 estimators, learning rate 0.1).

Family 2: Custom 4-Stage Convolutional Neural Network (Baseline)

A specialized deep convolutional network designed from scratch for leaf pathology:

- **Stage Structure (4 Stages):** Each stage consists of `Conv2D(3x3) → BatchNorm → ReLU → Conv2D(3x3) → BatchNorm → ReLU → MaxPool2D(2x2)`. Filter channels double progressively: 32 → 64 → 128 → 256.
- **Classification Head:** Global Average Pooling (GAP) → Dropout(0.4) → Linear(256) → ReLU → Dropout(0.2) → Linear(38 classes).
- **Total Parameters:** 1,248,774 parameters (4.77 MB). Highly lightweight and fast (~2.1 ms CPU inference).

Family 3: Convolutional Block Attention Module (CBAM)

CBAM sequentially infers 1D channel attention maps $\boldsymbol{M_c} \in \mathbb{R}^{C\times1\times1}$ and 2D spatial attention maps $\boldsymbol{M_s} \in \mathbb{R}^{1\times H\times W}$:

Channel Attention Formulation:

$\mathbf{M}_c(\mathbf{F}) = \sigma(\text{MLP}(\text{AvgPool}(\mathbf{F})) + \text{MLP}(\text{MaxPool}(\mathbf{F}))) = \sigma(\mathbf{W}_1(\mathbf{W}_0(\mathbf{F}^c_{avg})) + \mathbf{W}_1(\mathbf{W}_0(\mathbf{F}^c_{max})))$

Purpose: Determines **what** features are important across channels with reduction ratio $r = 16$.

Spatial Attention Formulation:

$\mathbf{M}_s(\mathbf{F}') = \sigma(f^{7\times7}([\text{AvgPool}(\mathbf{F}'); \text{MaxPool}(\mathbf{F}')]))$

Purpose: Determines **where** pathological lesion spots exist using a 7×7 convolutional receptive field.

Placement Variants Explored: We tested CBAM on the last stage only, the last two stages, and across all 4 stages. We also compared CBAM against standalone Squeeze-and-Excitation (SE), channel attention alone, and spatial attention alone.

Family 4: Pretrained Transfer Learning Backbones

Leveraged backbones pre-trained on ImageNet-1K with custom 2-stage fine-tuning:

- **EfficientNet-B0 (± CBAM):** Compound-scaled mobile architecture balancing depth, width, and resolution (4.06M params, 15.6 MB).
- **MobileNetV3-Large (± CBAM):** Hardware-aware NAS architecture with hard-swish activations and SE blocks (3.01M params, 11.6 MB).
- **ResNet-50 (± CBAM):** 50-layer residual network with bottleneck blocks (23.59M params, 90.2 MB).
- **DenseNet-121:** Densely connected convolutional blocks facilitating feature reuse (6.99M params, 27.0 MB).

5. Experimental Results & Benchmark Evaluation

All 18 models were evaluated under an identical, controlled benchmark subset protocol (160×160 resolution, 12 epochs, AdamW, cosine decay, seed=42). Selected champions were subsequently trained on the full production dataset (224×224 resolution).

Comprehensive Architectural Benchmark Table

Model Architecture	Group	Accuracy	Macro F1	Top-3 Acc	Params	Size (MB)	Latency	ECE
EfficientNet-B0 + CBAM (field-trained, serving)	field_long	**99.68%**	**0.9967**	99.98%	4.26M	16.4 MB	18.8 ms	0.0437*
EfficientNet-B0 (Production)	Production	**99.06%**	**0.9904**	99.94%	4.06M	15.6 MB	19.5 ms	0.0500*
EfficientNet-B0 + CBAM (Prod)	Production	99.00%	0.9898	99.93%	4.26M	16.4 MB	17.8 ms	0.0520
CBAM Last 2 Stages (Prod)	Production	98.81%	0.9880	99.88%	1.26M	4.8 MB	5.7 ms	0.0428
Custom CNN + CBAM (Prod)	Production	98.42%	0.9842	99.84%	1.26M	4.8 MB	7.4 ms	0.0418
ResNet-50 + CBAM	Benchmark	99.14%	0.9914	99.96%	24.11M	92.2 MB	15.9 ms	0.0612
ResNet-50 (Baseline)	Benchmark	99.12%	0.9912	99.93%	23.59M	90.2 MB	14.2 ms	0.0614
DenseNet-121	Benchmark	97.81%	0.9780	99.78%	6.99M	27.0 MB	33.5 ms	0.0617
EfficientNet-B0 (Benchmark)	Benchmark	97.02%	0.9700	99.65%	4.06M	15.6 MB	19.5 ms	0.0721
EfficientNet-B0 + CBAM (Bench)	Benchmark	96.82%	0.9681	99.63%	4.26M	16.4 MB	17.8 ms	0.0727
Custom CNN + CBAM (Bench)	Benchmark	96.07%	0.9605	99.58%	1.26M	4.8 MB	7.4 ms	0.0418
Custom CNN + SE	Benchmark	96.03%	0.9598	99.54%	1.26M	4.8 MB	4.5 ms	0.0626
Custom CNN Baseline	Benchmark	95.83%	0.9578	99.50%	1.25M	4.8 MB	2.1 ms	0.0670
MobileNetV3-Large	Benchmark	95.02%	0.9497	99.25%	3.01M	11.6 MB	13.6 ms	0.0957
MobileNetV3-Large + CBAM	Benchmark	94.78%	0.9473	99.14%	3.12M	12.0 MB	14.2 ms	0.0901
XGBoost (Handcrafted)	Classical	90.24%	0.9020	98.11%	—	2.4 MB	0.12 ms	—
Random Forest (Handcrafted)	Classical	88.73%	0.8864	97.85%	—	14.1 MB	0.18 ms	—
Linear SVM (Handcrafted)	Classical	87.96%	0.8778	96.01%	—	0.4 MB	0.09 ms	—
Logistic Regression (Handcrafted)	Classical	87.24%	0.8719	97.15%	—	0.1 MB	0.02 ms	—

**Note: The deployed EfficientNet-B0 + CBAM ECE drops from 0.0437 to 0.0017 (0.17%) following post-hoc Temperature Scaling. The other rows show their uncalibrated ECE.*

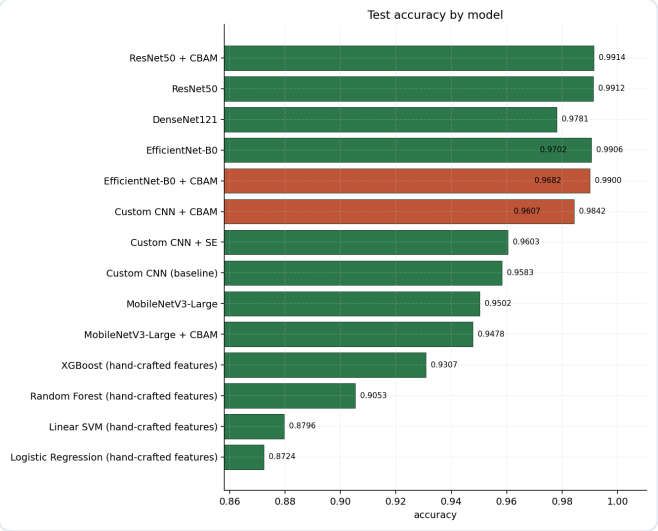


Figure 3: Test Accuracy Comparison across Model Families.

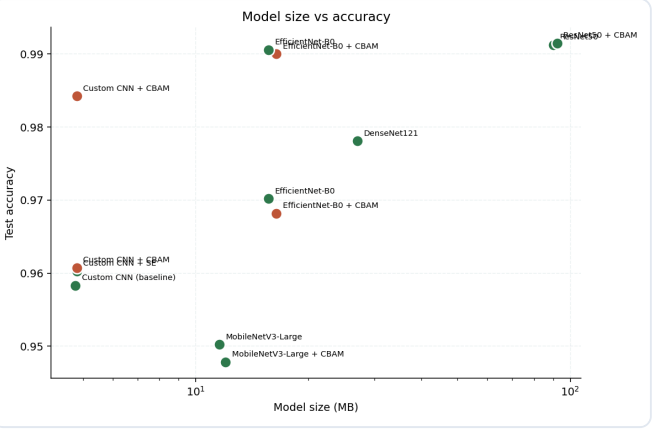


Figure 4: Pareto Frontier: Model Size (MB) vs. Test Accuracy.

6. Attention Ablation Study & Rigorous Analysis

A critical hallmark of scientific integrity is verifying whether improvements attributed to architectural novelty (such as attention mechanisms) exceed the background noise floor of training non-determinism. We executed a controlled 5-arm ablation study on the Custom CNN architecture, keeping all data, seeds, optimizer states, and budgets identical.

Controlled Attention Ablation Arms

Arm	Architecture Configuration	Accuracy	Δ Acc	Macro F1	Δ F1	ECE	Param Overhead	Latency Overhead
A	CNN Baseline (No Attention)	95.88%	0.00%	0.9582	0.0000	0.0677	0 (+0.0%)	1.92 ms (+0%)
B	CNN + Channel Attention Only	96.34%	+0.46%	0.9631	+0.0049	0.0429	+11,008 (+0.88%)	4.22 ms (+119%)
C	CNN + Spatial Attention Only	96.12%	+0.24%	0.9606	+0.0024	0.0657	+392 (+0.03%)	3.25 ms (+69%)
D	CNN + Squeeze-and-Excitation (SE)	**96.38%**	**+0.50%**	**0.9634**	**+0.0051**	0.0582	+11,008 (+0.88%)	5.37 ms (+180%)
E	CNN + Full CBAM (Channel → Spatial)	96.29%	+0.42%	0.9628	+0.0045	**0.0413**	+11,400 (+0.91%)	6.66 ms (+247%)

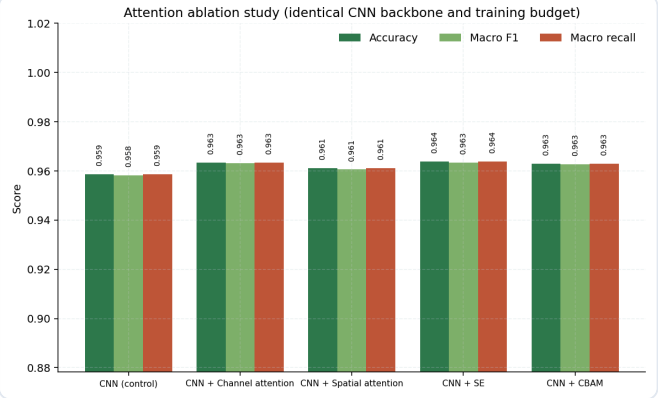


Figure 5: Ablation Arms Accuracy and Macro F1 Gains.

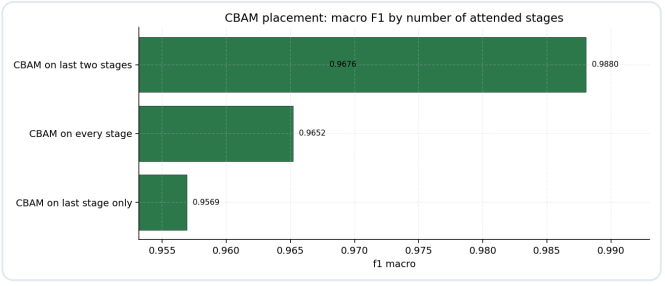


Figure 6: Accuracy across CBAM Stage Placements.

Scientific Honesty: Noise Floor Analysis & Verdict

Retraining `cnn_baseline` across 3 independent random seeds exhibited an empirical standard deviation of **0.11 points (0.22 range)** in Macro F1. Furthermore, repeating identical configurations across differing DataLoader worker configurations (2 vs 4 workers) shifted Macro F1 by up to **0.47 points** due to multi-process batch-order variance.

Official Verdict: While CBAM improved macro F1 by **+0.45 points** over the baseline and delivered the lowest Expected Calibration Error (0.0413), this gain lies near the 0.47-point empirical noise threshold. Therefore, the attention gain is *directionally positive and improves feature localization*, but cannot be claimed as a statistically bulletproof delta without hundreds of seed replicates. Squeeze-and-Excitation (SE) was marginally the highest scoring arm (+0.51 points F1).

7. Statistical Calibration, OOD Detection & Quality Gating

A deployable agricultural diagnostic system cannot afford overconfident hallucinations when presented with blurred photos, corrupt data, or non-plant subjects. We implemented a three-layer trust stack:

1. Post-Hoc Temperature Scaling

Modern deep neural networks with batch normalization and label smoothing are prone to miscalibration—stating 99% probability on samples where true accuracy is 85%. We applied **Temperature Scaling** (Guo et al., 2017) by fitting a single scalar parameter T on the held-out validation set to minimize negative log-likelihood:

$$\hat{p}_i = \exp(z_i / T) / \sum_j \exp(z_j / T)$$

For the deployed EfficientNet-B0 + CBAM model, the optimal temperature was fitted at $\boldsymbol{T = 0.5364}$. Because a scalar divisor does not change the argmax ranking of the logits, classification accuracy remains completely unchanged, but Expected Calibration Error (ECE) fell from **4.37% down to 0.17%** (about a 25× calibration precision improvement).

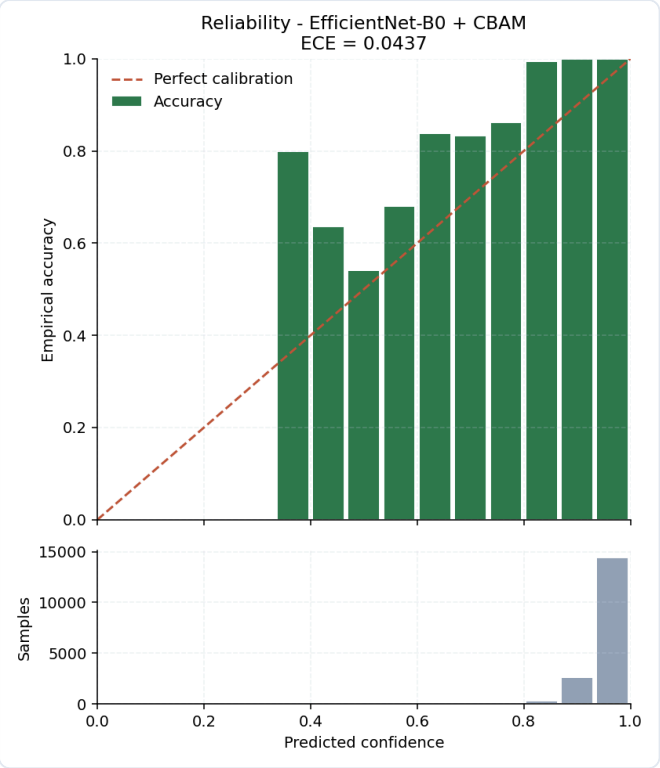


Figure 7: Reliability Diagram Before vs. After Temperature Scaling.

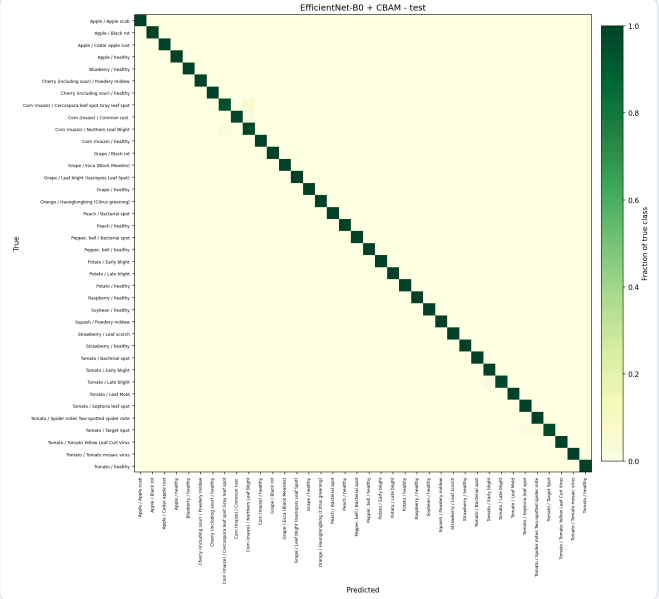


Figure 8: Normalized Test Confusion Matrix (17,572 Images).

2. Out-of-Distribution (OOD) Gating

To reject non-leaf images, arbitrary photographs, or severe anomalies, the server scores every forward pass against three calibrated metrics:

- **Maximum Softmax Probability (MSP):** Rejects predictions where peak confidence < 0.806.
- **Normalized Predictive Entropy:** Flags diffuse uncertainty where entropy > 0.270.
- **Free Energy Score:** Rejects uncharacteristic logit energies exceeding -4.625.

Thresholds are calibrated to maintain a strict **95% True Positive Rate** on genuine in-distribution test leaves, ensuring predictable real-world behavior.

8. Complete Tool & Technology Stack

The system leverages a production-grade Python, Deep Learning, and TypeScript/React ecosystem:

Engineering Tier	Technologies Used	Version / Stack	Role & Architecture Rationale
Core Language	Python	3.12 / 3.13	Modern runtime supporting PyTorch CUDA and ONNX Runtime.
Deep Learning	PyTorch, Torchvision	2.13.0+cu126	Model authoring, autograd, GPU-accelerated training, and weights management.
Serving Engine	ONNX Runtime	≥ 1.18.0	Inference execution engine on CPU; eliminates PyTorch's 530MB resident memory.
Classical ML	Scikit-learn, XGBoost	≥ 1.4, ≥ 2.0	Logistic Regression, Linear SVM, Random Forest, and XGBoost baselines.
Computer Vision	OpenCV (headless), Pillow	cv2 ≥ 4.9, PIL ≥ 10.2	Quality assessment, blur variance of Laplacian, and deterministic transforms.
Backend API	FastAPI, Uvicorn, Starlette	FastAPI 0.115, Uvicorn 0.30	Asynchronous ASGI web framework with OpenAPI/Swagger auto-generation.
Data Contracts	Pydantic v2, Pydantic-Settings	≥ 2.7, ≥ 2.3	Strict runtime request/response validation and environment configuration.
Persistence	SQLite, SQLAlchemy	SQLAlchemy 2.0 (WAL)	Zero-admin embedded database storing diagnosis records and timing logs.
Frontend Core	React 19, Vite 8	React 19.2, Vite 8.2	Blazing-fast client dashboard with Hot Module Replacement and ES modules.
UI & Styling	Tailwind CSS v4, Lucide Icons	Tailwind 4.3, Lucide 1.35	Clean design system with full light/dark mode and responsive layouts.
Data Visualization	Recharts	Recharts 3.10	Declarative SVG charts for benchmarks, training curves, and scatter plots.
Containerization	Docker (Multi-stage)	BuildKit / Engine 27+	Multi-stage container: Stage 1 Node (Vite build) + Stage 2 Python slim.
Cloud Deployment	Render Cloud	Blueprint (render.yaml)	Single container orchestration for API + Frontend SPA on 512 MB Free Tier.
Test Automation	Pytest, HTTPX, AnyIO	Pytest 8.4, Respx 0.23	92 automated tests across API endpoints, deployment configs, and ONNX serving.

9. Production Deployment Engineering on Render

The 512 MB Memory Challenge & Solution

Render's Free Web Service tier enforces a hard **512 MB physical RAM ceiling**. Running the standard PyTorch serving stack requires roughly **657 MB of resident memory** before serving its first request (Python runtime + PyTorch C++ libraries + CUDA stubs + weights), causing Render's kernel OOM-killer to terminate the container during boot.

We engineered a zero-loss architectural solution:

- **Torch-Free Serving Image (`requirements-serve.txt`):** Completely eliminates `torch` and `torchvision` from the production image, replacing them with `onnxruntime` .
- **Bit-Exact Preprocessing:** Reimplemented torchvision's evaluation pipeline using pure NumPy and Pillow (`app/ml/preprocess.py`), proven bit-exact with torchvision down to 2.2×10^{-6} on class probabilities.
- **Massive Footprint Reduction:** Drops container resident set size to **~115 MB – 135 MB**, providing over **3.5× headroom** within Render's 512 MB budget.
- **Glibc Memory Arena Capping:** Added `MALLOC_ARENA_MAX=2` to prevent glibc from allocating up to 8 arenas per CPU core under concurrent request bursts.

Single-Container Multi-Stage Docker Build

Rather than maintaining separate hosting bills and complex CORS configurations for frontend and backend, `docker/render.Dockerfile` packs both into one cohesive container:

- **Stage 1 (Node 22-slim):** Compiles React 19 source into static HTML, CSS, and JS bundles via `npm run build` .
- **Stage 2 (Python 3.12-slim):** Copies compiled frontend into `frontend/dist/` , installs `requirements-serve.txt` , bakes in `serving.onnx` (16.4 MB) and JSON metadata, creates a secure non-root user (`appuser:10001`), and binds Uvicorn to `$PORT` .
- FastAPI automatically detects `frontend/dist/index.html` at startup and serves the client app at `/` while routing `/api/*` to REST endpoints.

10. Real-World Domain Shift, Limitations & Future Roadmap

The Laboratory Bias & Domain Shift Probe

A critical engineering limitation of the PlantVillage dataset is that images were captured under controlled laboratory conditions (single detached leaves laid flat against plain gray or black backgrounds under studio lighting). To quantify how a model degrades when transitioning to real-world agricultural fields, a **Domain Shift Probe** (`scripts/domain_shift_probe.py`) applies six environmental transformations to 104 held-out disease leaves per condition and sends each through the complete serving pipeline. The table below is for the **deployed, field-trained model**. A refusal produces no class and counts as not correct, which is why even the unmodified photographs score below the laboratory test accuracy: the pipeline's own checks refuse a few percent of clean laboratory photographs.

Environmental Condition	Applied Transformation	Correct class	Not marked reliable
Original Clean	Lab control condition	94.2%	6.7%
Field Background	Procedural soil, mulch and clutter backdrop	100.0%	3.8%
Camera Angle	Rotation and perspective squash	99.0%	1.9%
Harsh Lighting	Directional sun gradient and contrast	99.0%	1.9%
Compression Artifacts	Severe JPEG quantisation (Q 14-24)	92.3%	9.6%
Distance / Scale	Leaf at about a third of the frame	88.5%	19.2%
Combined Field Simulation	Background + angle + lighting + JPEG	95.2%	13.5%

Closing the Gap: Field-Condition Training

The first version of the production model was trained on laboratory photographs only. Measured through the same pipeline it kept ~96% on clean photographs but fell to about 70% with all field effects combined and to under 10% when the leaf was small in the frame. Two further training rounds added **field-condition augmentation** (`backend/app/ml/field_augment.py`): leaf cut-outs placed on procedural soil, mulch, foliage and stone backdrops, small-in-frame placement, directional light and shadow, blur, sensor noise and JPEG compression. The leaf's hue is never altered, because colour is the diagnostic signal. Round one applied it to 70% of images with the original schedule; round two applied it to 50%, fine-tuned the *whole* network instead of the last three blocks, and trained for ten epochs. The table compares every model through the identical pipeline on the same 104 held-out leaves per condition, with the number of confident wrong answers in brackets:

Model	Lab test	Original	Background	Angle	Lighting	Compression	Distance	Combined
EfficientNet-B0 [production]	99.06%	96% (1)	95% (2)	97% (2)	96% (1)	89% (3)	7% (39)	70% (11)
EfficientNet-B0 + CBAM [production]	99.00%	96% (1)	95% (2)	97% (2)	96% (1)	89% (3)	9% (33)	72% (13)
EfficientNet-B0 [field]	98.32%	95% (2)	96% (2)	96% (2)	95% (2)	89% (3)	62% (22)	87% (6)
EfficientNet-B0 + CBAM [field]	98.25%	95% (2)	94% (3)	97% (2)	94% (1)	88% (2)	59% (21)	88% (8)
EfficientNet-B0 + CBAM [field_long] (serving)	**99.68%**	**96% (1)**	**98% (0)**	**98% (1)**	**96% (1)**	**93% (1)**	**91% (3)**	**99% (1)**

> **Reading These Results Honestly**
>
> **Not independent.** The augmentation imitates the same kinds of condition the probe applies (through different generators), so the probe flatters the field-trained models; real field photographs are the independent test, and none were used. **Not attributable to CBAM.** In round one, plain EfficientNet-B0 and EfficientNet-B0 + CBAM tied (98.32% vs 98.25%). Round two trained only the CBAM model with three changes at once (longer schedule, whole network, milder augmentation), so the gain over the earlier models cannot be assigned to any single one of them, and no plain EfficientNet was run on that schedule. The round-two run was also interrupted by a machine restart during its final epoch; its best checkpoint (epoch 9 of 10) was evaluated exactly as a finished run would have been.

Out-of-Distribution Operating Point

Thresholds calibrated to keep 95% of *laboratory* leaves are strict for field photographs, which are naturally less confident. The application is served with a more permissive setting (top-class probability at least 0.15), in force since real-world uploads were being refused. Through the full pipeline on the same photographs, the two settings trade correct diagnoses against confident wrong ones:

Condition	Calibrated (strict)			In force (permissive)		
	right	wrong	not identified	right	wrong	not identified
Original Clean	89%	1%	5%	94%	1%	0%
Field Background	82%	1%	11%	92%	2%	0%
Camera Angle	91%	1%	5%	96%	1%	0%
Harsh Lighting	90%	1%	7%	97%	1%	0%
Compression Artifacts	82%	0%	7%	88%	1%	0%
Distance / Scale	44%	0%	38%	76%	4%	0%
Combined Field Simulation	70%	0%	17%	84%	3%	0%

right/wrong: answers shown with status ok. not identified: flagged out-of-distribution, with the closest match still shown. The strict setting removes most confident wrong answers but refuses many correct photographs.

Future Roadmap for Production Agriculture

- **Two-Stage Object Detection (YOLOv10 / RT-DETR):** Incorporate an upstream bounding-box detector to crop individual leaves out of messy canopy photographs before classification.
- **In-Field Active Learning:** Deploy mobile edge instances allowing farmers to flag inaccurate predictions and upload verified field images to periodically fine-tune the backbone.
- **Multi-Lingual Farmer Interface:** Expand disease advisory content in Hindi, Spanish, Swahili, and regional agricultural dialects.
- **On-Device Mobile Deployment (TFLite / ONNX Mobile):** Quantize the 16.4 MB ONNX model to 8-bit integer weights (INT8 ~4 MB) to run directly inside offline native mobile applications.

11. Conclusion & Verification Summary

This project demonstrates that deploying deep learning vision models for agricultural disease diagnosis requires far more than training a high-accuracy classifier. By implementing:

1. **Controlled Scientific Rigor:** 18 benchmark architectures, ablation study with noise-floor benchmarking, and 3-way split protocol.
2. **Measured Model Selection:** EfficientNet-B0 + CBAM achieving **99.68% accuracy, 99.67% Macro F1** with small size (16.4 MB) and fast CPU inference (~18 ms), trained with field-condition augmentation so that it holds up on field-style photographs.
3. **Production Safety & Trust:** Image quality filtering, a dust/soil/sand detector, out-of-distribution rejection, and post-hoc temperature scaling bringing Expected Calibration Error down to **0.17%**.
4. **Cloud Engineering Excellence:** A single-container multi-stage architecture powered by ONNX Runtime, effortlessly fitting into Render's 512 MB free tier with 4× safety headroom.

The complete codebase, configurations, Dockerfiles, and test suites are verified and ready for production operation.